

PRODUCT VISION BOARD & INTERNAL SOLUTION BRIEF

Guía de referencia, ejemplos y paso a paso para tu primer artefacto de producto.

AI for Developers — Cohorte 4 — Agosto-Octubre 2026

Danny Bravo - @Danny.Bravo.AI

[linkedin.com/in/dannybravo](https://www.linkedin.com/in/dannybravo)

Contenido

Cap. 1 — Product Vision Board (si construyes un producto propio o del banco)

Cap. 2 — Internal Solution Brief (si resuelves un problema de tu empresa)

Cap. 3 — Como usar tu Coding Agent para completar tu artefacto

Que template uso?

Tu caso	Template	Archivo
Idea propia / banco de productos / startup	Product Vision Board	hcai-c4-product-vision-board.md
Proyecto de tu empresa / solucion interna	Internal Solution Brief	hcai-c4-internal-solution-brief.md

Deadline: Estacion 2 — jueves 27 de agosto. Lleva tu artefacto listo y tus dudas al canal de la cohorte y a los instructores.

Product Vision Board

Para quienes construyen un producto propio, una startup o eligen del banco de productos.

1 PROBLEMA

Define el dolor específico que resuelves. Debe persistir aunque los modelos evolucionen. Evalúa su durabilidad con un score de 1 a 5.

Consejos:

- Buenos problemas son de **workflow/integración**, no de generación de outputs.
- Preguntate: este dolor existirá cuando salga GPT-6?
- Ejemplo bueno: "Doctores pasan 6h/semana en billing compliance".

Errores comunes:

- "Queremos usar AI para X" — eso no es un problema, es una solución.
- "La gente necesita resúmenes" — ya está commoditizado.

2 SEGMENTO TARGET

Define con precisión quien usa tu producto y quien puede vetar su adopción.

Consejos:

- Se específico: no "empresas", sino "equipos de legal en startups Series A-B".
- El veto holder es crucial — si no lo identificas, tu distribución falla.

Errores comunes:

- "Todo el mundo" como segmento.
- Ignorar al veto holder (ej: el CISO que bloquea herramientas AI).

3 MOAT PRIMARIO

Elige UNO: Data Moat (data única), Distribution Moat (canal embebido) o Trust Moat (reliability/compliance). Es tu ventaja defensible.

Consejos:

- **Data:** cada usuario genera data que mejora el producto (flywheel).
- **Distribution:** estás embebido donde los usuarios ya trabajan.
- **Trust:** auditorías, compliance, safety que toma meses construir.

Errores comunes:

- "Nuestra UX es mejor" — eso es un feature, no un moat.
- Elegir tres moats. Solo puedes defender uno como primario.

4 ARENA COMPETITIVA

Pioneer (mercado nuevo), Disruptor (10x mejor) o Enhancer (fortalecer existente).

Consejos:

- No puedes superar a un gigante a golpes. Superalo con estrategia.
- CodiumAI sobrevivio a Copilot enfocandose en testing, no en code gen.

Errores comunes:

- Competir cabeza a cabeza con Google/Microsoft sin diferenciador.

5 UX PARADIGM

Assistant (sugiere), Agent (ejecuta con limites), Autonomous (sin supervision) o Embedded (invisible).

Consejos:

- Un asistente medico: Assistant. Un detector de fraude: Autonomous.
- El paradigma define expectativas de control y autonomia del usuario.

Errores comunes:

- Autonomous sin justificar por que el humano no esta en el loop.

6 AI DECISION TRIANGLE

Cost, Capability o Speed — no puedes maximizar las tres.

Consejos:

- Copilot prioriza Speed. Harvey prioriza Capability.
- Tu eleccion impacta que modelos usas y como disenas tu arquitectura.

Errores comunes:

- "Las tres" — no es realista.

7 MODELO ECONOMICO

Elige el modelo de pricing y haz la matematica basica: costo por usuario, revenue por usuario, gross margin.

Consejos:

- Si AI costs > 30% del revenue por usuario, estas en danger zone.
- Modela economics a 100x escala ANTES de construir.

Errores comunes:

- Cobrar flat sin entender tus costos de inference.

8 METRICAS DE EXITO

2 metricas de usuario + 2 metricas especificas de AI.

Consejos:

- Deben ser medibles con un target concreto.
- Metricas AI detectan drift antes de que los usuarios lo noten.

Errores comunes:

- "Numero de usuarios" como unica metrica.

9 RIESGOS CRITICOS

Commoditizacion en 12 meses, replicabilidad en 6 semanas, y como se rompe la confianza a escala.

Consejos:

- Si no puedes articular como fallas a escala, no estas listo.
- "Pivotearemos" no es estrategia contra commoditizacion.

Errores comunes:

- Dejar estas respuestas vacias o genericas.
-

EJEMPLO: PRODUCT VISION BOARD

Producto 4 del banco: Sistema de Escrow Autonomo para AI (AlsaEscrow)

1 PROBLEMA

Problema:

Los rieles de pago tradicionales no soportan comercio entre agentes de AI. Micropagos M2M son inviables con comisiones estandar (2.9% + \$0.30/transaccion). No existe mecanismo de confianza para pagos entre agentes sin intervencion humana.

Durabilidad:

Si — problema de WORKFLOW/INTEGRACION.
Los modelos foundation no resuelven infraestructura de pagos.

Durability Score:

5

2 SEGMENTO TARGET

Segmento:

Developers y empresas Web3/Fintech que construyen aplicaciones con agentes autonomos que necesitan transaccionar entre si (B2B).

Veto holder:

CTOs y compliance de empresas Fintech.
En Web3: auditores de smart contracts.

3 MOAT PRIMARIO

Moat:

Trust Moat

Justificacion:

Contratos inteligentes auditados e inmutables.

Transacciones verificables on-chain.

"Know Your Agent" (KYA) para identidad de agentes.

La confianza depende de código auditado, no de una empresa.

4 ARENA COMPETITIVA

Arena:

Pioneer (AI-Native) — el mercado de pagos entre agentes autónomos no existía antes de AI.

vs. Gigantes:

Stripe/PayPal no diseñados para M2M.

Mastercard Agent Pay usa rieles tradicionales.

Enfoque nativo blockchain + AI: nicho complementario.

5 UX PARADIGM

Paradigma:

Autonomous — opera sin supervisión humana.

Por que:

En transacciones M2M de alta frecuencia,

cualquier fricción humana rompe la propuesta de valor.

La autonomía ES el producto.

6 AI DECISION TRIANGLE

Prioridad:

Speed — liquidacion en segundos, no dias.

Trade-offs:

Acepto mayor costo de gas a cambio de velocidad.

Evaluator (LLM) no necesita ser el mas potente — la validacion es binaria (cumplio/no cumplio).

7 MODELO ECONOMICO

Pricing:

Usage-Based — comision 0.3-0.5% por transaccion (vs. 2.9% tradicional).

Escala 10x:

Si — cada transaccion genera revenue.

Costo/tx:

\$0.02 (gas + inference)

Revenue/tx (\$10):

\$0.04 (0.4%)

Gross margin:

50%

8 METRICAS

Usuario:

1. Volumen de transacciones/mes
2. Tiempo de liquidacion (target: <30s)

AI:

1. Validacion correcta del evaluador (target: >98%)
2. Disputas con intervencion manual (target: <2%)

Commoditizacion:

Bajo. Requiere integracion profunda con protocolos blockchain. Estandares emergiendo — quien construya primero define el estandar.

Replicabilidad:

Parcial. Smart contract base replicable, pero trust moat (auditorias, red KYA) toma meses.

Ruptura trust:

Bug en smart contract = perdida de fondos.

Mitigacion: auditorias, monitoreo on-chain, limites de transaccion progresivos.

CAPITULO 2

Internal Solution Brief

Para quienes vienen de empresas y van a resolver un problema interno de su organizacion con AI.

Si tu empresa te envio al programa, tu objetivo no es crear una startup — es construir una solucion que resuelva un dolor real de tu organizacion. Este artefacto te ayuda a definir el problema, los stakeholders, las restricciones y el enfoque tecnico de forma que tu sponsor y tu equipo lo entiendan.

Como se diferencia del Product Vision Board?

Product Vision Board	Internal Solution Brief
Mercado target y clientes	Stakeholders internos y sponsor
Revenue y modelo de negocio	Ahorro de costos / eficiencia
Landscape competitivo	Proceso interno actual
Vision y posicionamiento	Estado futuro deseado
Tu eliges a tus usuarios	Los usuarios ya estan definidos
Pregunta: debemos construir esto?	Pregunta: como lo resolvemos bien?

1 PROBLEMA DE NEGOCIO

Describe el dolor que existe hoy en tu organizacion. Se especifico y cuantifica: tiempo perdido, errores, costos.

Consejos:

- Cuantifica siempre. "Es lento" no sirve. "Toma 5 dias y genera 3 errores/semana" si.
- Pregunta a tu sponsor: cual es el costo real de este problema?
- Si el problema no tiene costo medible, probablemente no es prioritario.

Errores comunes:

- "Queremos modernizarnos" — eso no es un problema, es un deseo.
- No cuantificar el impacto.

2 STAKEHOLDERS Y SPONSOR

Identifica al sponsor (quien aprueba), los usuarios finales (quien lo usa) y los bloqueadores potenciales (IT, seguridad, compliance).

Consejos:

- El sponsor no es necesariamente tu jefe — es quien tiene autoridad sobre el presupuesto.
- Habla con los bloqueadores ANTES de construir, no despues.

Errores comunes:

- Construir sin sponsor claro = solucion que nadie adopta.

3 ESTADO ACTUAL

Documenta como se resuelve el problema hoy: proceso, herramientas, workarounds. Identifica que funciona bien (no tocar) y que no funciona (oportunidad).

Consejos:

- Observa el proceso real, no el proceso documentado.
- Los workarounds son oro: revelan necesidades que nadie formalizan.
- Lo que funciona bien es tan importante como lo que no funciona.

Errores comunes:

- Asumir que sabes como funciona sin preguntar.

4 ESTADO FUTURO DESEADO

Describe como se veria el proceso si tu solucion funciona perfectamente. Que cambia para el usuario final en su día a día?

Consejos:

- Se concreto: "El analista recibe el reporte listo en 10 minutos en lugar de armarlo en 4 horas".
- Piensa en el impacto operativo, no en la tecnologia.

Errores comunes:

- "Usaremos AI" — eso es la herramienta, no el resultado.

5 CRITERIOS DE EXITO

Define 2-3 metricas medibles que demuestren que la solucion funciona. Deben ser metricas que tu sponsor entienda.

Consejos:

- Formato: Metrica | Valor actual | Target.
- Ejemplo: "Tiempo de aprobacion | 5 dias | 1 dia".
- Las metricas deben ser verificables con datos reales.

Errores comunes:

- Metricas tecnicas que el sponsor no entiende (ej: latencia p99).

6

RESTRICCIONES

Identifica restricciones técnicas (stack, integraciones), de datos (acceso, privacidad), organizacionales (presupuesto, timeline) y de compliance/seguridad.

Consejos:

- Si no conoces una restricción, pregunta a tu sponsor. Es tu tarea esta semana.
- Las restricciones de datos son las más subestimadas y las que más proyectos matan.

Errores comunes:

- Ignorar restricciones de seguridad y descubrirlas al final.

7

ENFOQUE TECNICO

Que capacidad de AI aplica y por que AI es la solución correcta (y no una automatización tradicional). Arquitectura de alto nivel.

Consejos:

- Si una regla de negocio simple resuelve el problema, AI es overkill.
- AI brilla en: clasificación, extracción, generación contextual, orquestación.

Errores comunes:

- "Porque AI está de moda" no es una justificación técnica.

8

RIESGOS Y DEPENDENCIAS

Identifica los 2 riesgos principales con su mitigación, y las dependencias externas (APIs, accesos, aprobaciones).

Consejos:

- Los riesgos de adopción (change management) matan más proyectos que los técnicos.
- Cada riesgo debe tener una mitigación concreta.

9

LIMITES DE ALCANCE

Define que SI construyes en 6 semanas y que NO construyes ahora. Esto protege tu timeline y las expectativas de tu sponsor.

Consejos:

- Se explícito: "NO incluye integración con SAP en esta fase".
- Menos alcance = más calidad. El Demo Day premia productos que funcionan, no features.

Errores comunes:

- No definir límites y terminar con un MVP que no hace nada bien.
-

EJEMPLO: INTERNAL SOLUTION BRIEF

Caso: Automatización de reportes de compliance para equipo financiero

1 PROBLEMA DE NEGOCIO

Problema:

El equipo de compliance financiero dedica 12 horas/semana a compilar reportes regulatorios manualmente. Extraen datos de 4 sistemas, los cruzan en Excel y redactan narrativas.

Costo:

3 analistas x 12h/semana = 36h/semana.
~\$4,300 USD/mes en horas productivas perdidas.
2-3 errores por reporte que generan observaciones del regulador.

Desde cuando:

Siempre — el equipo creció pero el proceso no cambió.

2 STAKEHOLDERS

Sponsor:

CFO — tiene presupuesto y autoridad para cambiar proceso.

Usuarios finales:

3 analistas de compliance que arman los reportes semanalmente y el gerente que los revisa.

Bloqueadores:

CISO (política de datos sensibles fuera de la org),
equipo de IT (integraciones con sistemas internos).

3 ESTADO ACTUAL

Proceso hoy:

1. Extraer datos manualmente de 4 sistemas (SAP, CRM, banco, hojas internas)
2. Cruzar en Excel con reglas manuales
3. Redactar narrativa del reporte
4. Revision del gerente
5. Envio al regulador

Funciona bien:

La revision del gerente — agrega criterio humano valioso.

No funciona:

Extraccion manual (lenta y propensa a errores),
cruce en Excel (fragil), redaccion repetitiva.

4 ESTADO FUTURO

Vision:

El agente extrae datos automaticamente de los 4 sistemas,
cruza la informacion, genera el borrador del reporte con
narrativas estandar, y lo presenta al gerente para revision.
El proceso pasa de 12 horas a 1 hora.

Cambio diario:

Los analistas pasan de compilar datos a validar
y analizar — trabajo de mayor valor.

5 CRITERIOS DE EXITO

Metricas:

1. Tiempo de compilacion: 12h -> 1h (target)
2. Errores por reporte: 2-3 -> 0 (target)
3. Satisfaccion del equipo (encuesta): 3/10 -> 8/10

6

RESTRICCIONES

Técnicas:

SAP on-premise, acceso via API limitada.

CRM en Salesforce (API disponible).

Datos:

Datos financieros sensibles — no pueden salir de infra de la empresa. LLM debe correr on-premise o usar API con acuerdos de procesamiento de datos.

Organizacionales:

Presupuesto aprobado para MVP.

Timeline: 6 semanas para prototipo funcional.

Compliance:

El reporte final SIEMPRE debe ser revisado por humano antes de envío al regulador (no negociable).

7

ENFOQUE TECNICO

Capacidad AI:

Extraccion de datos (conectores API + scraping),
cruce automatizado (reglas + validacion AI),
generacion de narrativa (LLM con template).

Por que AI:

La redaccion de narrativas requiere contexto variable y lenguaje natural — no es automatizable con reglas.
El cruce de datos tiene excepciones que un LLM maneja mejor que un script rigido.

8 RIESGOS

Riesgo 1:

Acceso a API de SAP limitado.

Mitigacion: exportar CSV como alternativa temporal.

Riesgo 2:

Alucinacion en narrativa del reporte.

Mitigacion: revision humana obligatoria + checks automaticos contra datos fuente.

Dependencias:

Aprobacion del CISO para uso de API de LLM con datos financieros.

9 ALCANCE

En alcance:

- Extraccion automatica de 2 de 4 sistemas (CRM + banco)
- Cruce de datos con reglas basicas
- Generacion de borrador de reporte
- Interfaz de revision para el gerente

Fuera de alcance:

- Integracion con SAP (fase 2)
- Envio automatico al regulador
- Dashboard historico de reportes

Como usar tu Coding Agent

Paso a paso para completar tu artefacto con Claude Code u otro agente de código.

Tu template de Markdown no es solo un documento — es el input para tu Coding Agent. El agente te va a ayudar a investigar, ser crítico con tus respuestas y asegurar que la calidad de tu artefacto sea alta.

1 PREPARA TU ENTORNO

Crea una carpeta para tu proyecto y copia el template de Markdown:

```
mkdir mi-producto && cd mi-producto
cp /ruta/hcai-c4-product-vision-board.md .
# o si eres corporativo:
cp /ruta/hcai-c4-internal-solution-brief.md .
```

Tip: si ya tienes notas, investigacion o documentos relevantes, ponlos en la misma carpeta como archivos .md o .txt. El agente los leera como contexto.

2 ABRE UNA SESION CON TU CODING AGENT

Abre Claude Code (o tu agente preferido) en la carpeta de tu proyecto:

```
c laude
```

Otros agentes compatibles: Cursor, Windsurf, Codex, GitHub Copilot Agent. Lo importante es que el agente pueda leer y editar archivos en tu carpeta.

3 PIDE AYUDA PARA INVESTIGAR

Dale al agente tu template y pidele que te ayude a investigar. Ejemplo:

Si tienes idea propia:

Prompt:

Lee mi product-vision-board.md. Mi idea es [descripcion]. Ayudame a investigar: genera un deep research que valide esta idea y otro que la critique. Guarda ambos como archivos .md.

Si eres corporativo:

Prompt:

Lee mi internal-solution-brief.md. El problema en mi empresa es [descripcion]. Investiga como otras organizaciones han resuelto problemas similares con AI y que riesgos enfrentaron. Guarda la investigacion como archivo .md.

Si elegiste del banco:

Prompt:

Lee mi product-vision-board.md y el banco-de-productos.md. Elegi el producto [N]. Investiga el estado del mercado, competidores, y validacion de la oportunidad. Guardalo en .md.

4

LLENA TU TEMPLATE CON EL AGENTE

Ahora pidele al agente que te ayude a llenar cada seccion del template. Lo mas importante: pidele que sea critico.

Prompt clave:

Prompt:

Ahora ayudame a llenar mi [product-vision-board.md / internal-solution-brief.md] seccion por seccion.

Para cada seccion:

1. Proponme un borrador basado en la investigacion
2. Se MUY CRITICO con lo que escribo — si algo es vago, generico o debil, dimelo y propon algo mejor
3. Escribe la version final en el archivo .md

Empecemos por la seccion 1.

5

VALIDA Y REFINA

Cuando tengas el template completo, pidele al agente una revision final:

Prompt de validacion:

Prompt:

Revisa mi [template].md completo como si fueras un inversor / sponsor exigente. Para cada seccion:

- Es lo suficientemente especifico?
- Tiene datos que lo respalden?
- Hay contradicciones entre secciones?

Dame una calificacion de 1-5 por seccion y sugiere mejoras concretas para las que esten debajo de 4.

6 LLEGA LISTO A LA ESTACION 2

Al final debes tener en tu carpeta:

- **Tu template completo** (product-vision-board.md o internal-solution-brief.md)
- **Deep research de validacion** (archivo .md)
- **Deep research de critica / riesgos** (archivo .md)
- **Notas adicionales** (entrevistas, datos, contexto)

Todo esto es el input para la Estacion 2, donde profundizaremos en descubrimiento, job-to-be-done y co-crearemos tu PRD.

CHECKLIST DE ENTREGA

- Template seleccionado (PVB o Internal Solution Brief)
- Deep research de validacion completado con tu Coding Agent
- Deep research de critica / riesgos completado
- Template llenado seccion por seccion con revision critica del agente
- Validacion final del agente (score 4+ en cada seccion)
- Listo para presentar en la Estacion 2 (jueves 27 de agosto)

AI for Developers | 30X — Cohorte 4 — Agosto-Octubre 2026 — Danny Bravo — linkedin.com/in/dannybravo